

PROVA G1 – TIPO 2

Gabarito

Questão Resp.

01	B
02	B
03	B
04	B
05	C
06	B
07	C
08	D
09	E
10	A
11	B
12	C
13	B
14	B
15	C
16	OCP

1 – B.

I, II e III estão corretas. A IV é falsa porque programação imperativa também utiliza funções.

2 – B.

Controle de acesso ao estado interno → encapsulamento.
Redefinição de comportamento herdado → polimorfismo.

3 – B.

Fatos + regras + consultas → mecanismo de **inferência lógica**.

4 – B.

Uma linguagem multiparadigma pode combinar recursos de diferentes paradigmas no mesmo projeto.

5 – C.

A função não modifica a entrada e produz o mesmo resultado para a mesma entrada → função pura + imutabilidade + transformação de dados.

6 – B.

I, II e III estão corretas. A IV é falsa: muitas responsabilidades indicam principalmente problema de coesão; não significa necessariamente baixo acoplamento.



7 – C.

Dividir responsabilidades relacionadas em classes específicas busca **aumentar a coesão**.

8 – D.

Interface obriga classes a implementar métodos que não utilizam → **ISP**.

9 – E.

Fazer `Pedido` depender de `Repositorio`, e não de `RepositorioMySQL`, reduz a dependência direta de uma implementação concreta → **DIP**.

10 – A.

- diferentes motivos para alteração → SRP
- subclasse não substitui adequadamente a base → LSP
- interface com métodos não utilizados → ISP
- nova funcionalidade exige modificar classe existente → OCP

11 – B.

I, II e III estão corretas. Microservices não eliminam infraestrutura.

12 – C.

Cada serviço controla seus próprios dados e utiliza comunicação entre serviços quando precisa de informações de outro domínio.

13 – B.

A tabela de histórico do Flyway registra quais migrações foram executadas.

14 – B.

I, II e III estão corretas. O mesmo código pode ser executado com configurações diferentes.

15 – C.

Centralizar configurações não significa que elas serão automaticamente atualizadas de forma dinâmica em todos os serviços. Isso depende da solução adotada.

Questão 16 – Tipo 2 — OCP

A questão apresenta uma `CalculadoraArea` que utiliza `instanceof` para identificar círculo e retângulo e que precisaria ser modificada quando surgisse um triângulo.

a) Resposta esperada

O princípio violado é o **OCP — Open/Closed Principle**.

A classe `CalculadoraArea` precisa ser modificada sempre que uma nova forma geométrica é adicionada.



Por exemplo, para adicionar `Triangulo`, seria necessário inserir uma nova condição:

```
if (forma instanceof Triangulo) {  
    ...  
}
```

Isso significa que a classe não está adequadamente aberta para extensão, pois novas funcionalidades exigem alterações no código existente.

b) Solução

Criar uma abstração:

```
interface Forma {  
    double calcularArea();  
}
```

Implementações:

```
class Circulo implements Forma {  
    @Override  
    public double calcularArea() {  
        return Math.PI * raio * raio;  
    }  
}  
class Retangulo implements Forma {  
    @Override  
    public double calcularArea() {  
        return largura * altura;  
    }  
}
```

E:

```
class Triangulo implements Forma {  
    @Override  
    public double calcularArea() {  
        return base * altura / 2;  
    }  
}
```

A calculadora fica:

```
class CalculadoraArea {  
    public double calcularArea(Forma forma) {  
        return forma.calcularArea();  
    }  
}
```

Agora, para adicionar:

```
class Pentagono implements Forma {
```



```
} ...
```

a `CalculadoraArea` **não precisa ser alterada.**

Explicação

Antes:

```
CalculadoraArea
|
+-- if Circulo
+-- if Retangulo
+-- if Triangulo
+-- if Pentagono
+-- ...
```

Depois:

```
          Forma
          |
+-----+-----+
|         |         |
Circulo  Retangulo  Triangulo
```

A calculadora conhece apenas `Forma`.

Esse é exatamente o espírito do **OCP**:

Aberto para extensão, fechado para modificação.